

Lab Report 1 - Cybersecurity Analytics

**Analyzing a Simple Packet Capture in Wireshark**

## Introduction

This lab is designed to work with VM machines and gain practical experience with packet analysis using Wireshark. Wireshark is used to examine the network traffic and analysis the packets. Packet analysis is typically performed by a packet sniffer, a tool used to capture raw network data going across the wire (Sanders, 2017). The packet analysis can be found for free and commercial use. Most popular packet analysis programs are Wireshark, Omnipcap (GUIs), and tcpdump (Command line program). Packet analyzer has the ability to grab the raw packet from the wire, wireless, Bluetooth, VLAN, PPP, and other network types, without getting processed by the application (Nath, 2015).

Wireshark uses *pcap* and *libpcap* files to capture packets, and it can capture live traffic and offline traffic. Network analysis provides valuable details for network security and troubleshooting. By analyzing packets, security teams can get a better understanding of the network and its behavior. It can be examined in detail of the packets in transport layer and application layer protocols including TCP and HTTP. According to the Sanders Although most of this book focuses on using packet analysis for network troubleshooting, a considerable amount of real-world packet analysis is done for security purposes (2017).

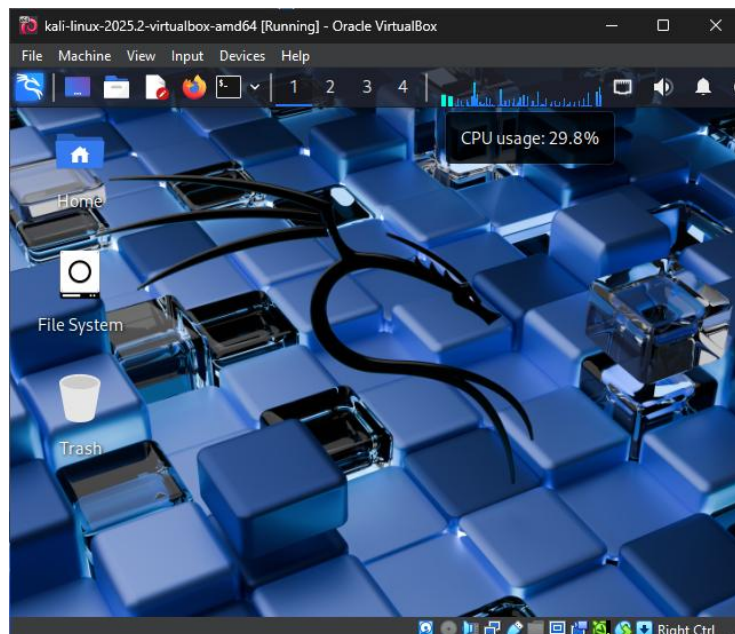
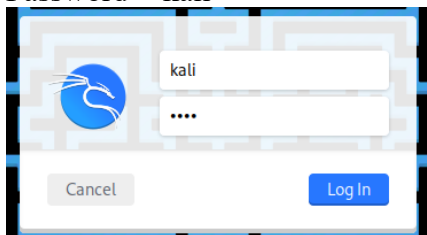
This lab works through the Kali Linux environment and gathers information on HTTP and TCP packets using Wireshark and its inbuilt filters, to get a better understanding of the packets.

1. Logged to Azure Lab environment using following web link and logged into the Kali VM using Oracle VirtualBox.

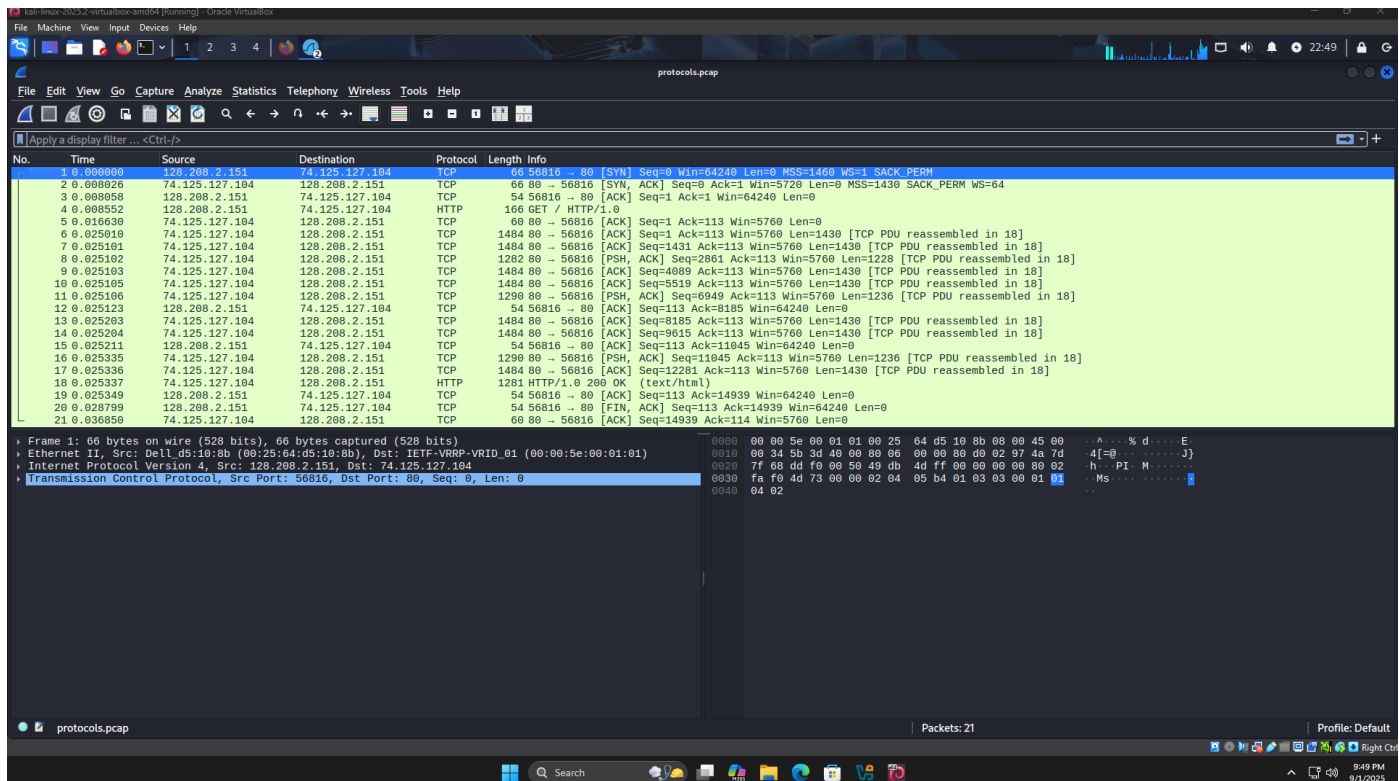
[https://labs.azure.com/virtualmachines?feature\\_vnext=true](https://labs.azure.com/virtualmachines?feature_vnext=true)

Username - kali

Password - kali

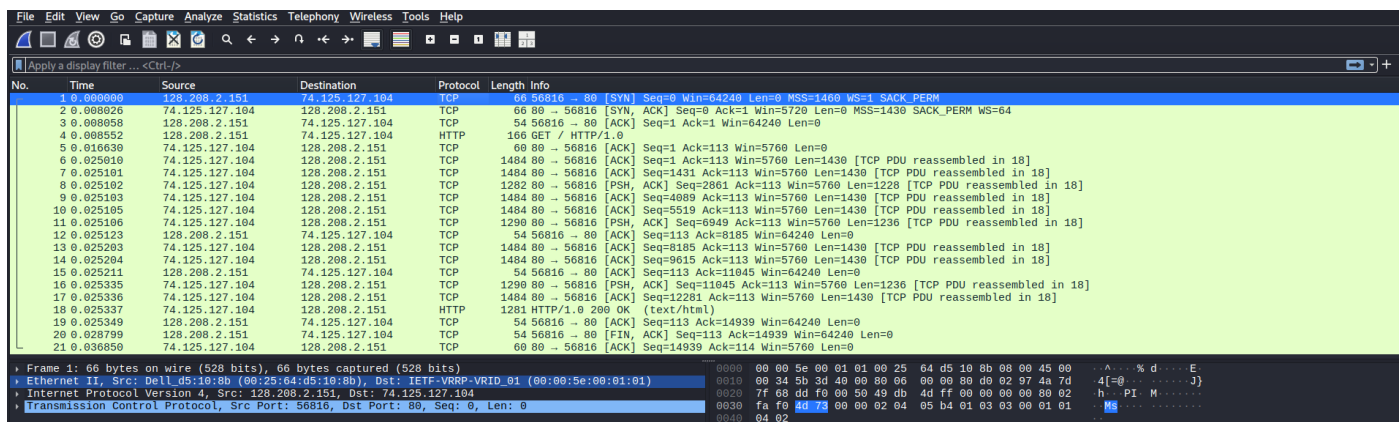


- Opened the Wireshark and then opened the provided *protocols.pcap* file to analyze the packet.



- Selected the TCP protocol from the *protocols.pcap* file and discovered the output details.

This represents the Transmission Control Protocol (TCP). TCP included in the Transport Layer protocol and it has responsibility to establish a connection between two endpoints. The source IP address 128.208.2.151 from source port 56816 initiates the connection to reach to the destination point of Google webserver IP address 74.125.127.104. The selected TCP has a SYN flag, the first stage of the TCP three-way handshake, and this indicates the source or host requested to establish a new connection. This traffic is encapsulated in IPv4 packets. Total frame size is 66 bytes.



4. Selected the HTTP protocol from the *protocols.pcap* file and discover the output details.

The capture file *protocols.pcap* shows the HyperText Transfer Protocol (HTTP) communication using TCP as the transport layer protocol. HTTP is an application layer protocol. The communication begins with a three-way handshake between the client IP address 128.208.2.151 using source port 56816 and the destination google webserver IP address 74.125.95.104 using HTTP port 80. This traffic is encapsulated in IPv4 packets. Here, the connection is set up. Then the HTTP GET request is transmitted. Total frame size is 166 bytes.

The screenshot shows the Wireshark interface with a packet list on the left and a packet details pane on the right. The packet list shows several packets, with the selected packet being a TCP packet (No. 4) from 128.208.2.151 to 74.125.127.104, Seq=0, Win=64240, Len=0, MSS=1460, SACK\_PERM. The packet details pane shows the Ethernet II, Internet Protocol Version 4, and Transmission Control Protocol layers. The TCP layer shows Seq=0, Win=64240, Len=0, MSS=1460, SACK\_PERM, and the HTTP layer shows GET / HTTP/1.0.

| No. | Time     | Source         | Destination    | Protocol | Length | Info                                                                                  |
|-----|----------|----------------|----------------|----------|--------|---------------------------------------------------------------------------------------|
| 1   | 0.000000 | 128.208.2.151  | 74.125.127.104 | TCP      | 60     | 56816 → 80 [SYN, Seq=0 Win=64240 Len=0 MSS=1460 WS=1 SACK_PERM                        |
| 2   | 0.000026 | 74.125.127.104 | 128.208.2.151  | TCP      | 66     | 80 → 56816 [SYN, ACK] Seq=0 Ack=1 Win=5720 Len=0 MSS=1430 SACK_PERM WS=64             |
| 3   | 0.000058 | 128.208.2.151  | 74.125.127.104 | TCP      | 54     | 56816 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0                                          |
| 4   | 0.000552 | 128.208.2.151  | 74.125.127.104 | HTTP     | 166    | GET / HTTP/1.0                                                                        |
| 5   | 0.016030 | 74.125.127.104 | 128.208.2.151  | TCP      | 60     | 80 → 56816 [ACK] Seq=1 Ack=113 Win=5760 Len=0                                         |
| 6   | 0.025010 | 74.125.127.104 | 128.208.2.151  | TCP      | 1484   | 80 → 56816 [ACK] Seq=1 Ack=113 Win=5760 Len=1430 [TCP PDU reassembled in 18]          |
| 7   | 0.025101 | 74.125.127.104 | 128.208.2.151  | TCP      | 1484   | 80 → 56816 [ACK] Seq=1431 Ack=113 Win=5760 Len=1430 [TCP PDU reassembled in 18]       |
| 8   | 0.025102 | 74.125.127.104 | 128.208.2.151  | TCP      | 1282   | 80 → 56816 [PSH, ACK] Seq=2861 Ack=113 Win=5760 Len=1228 [TCP PDU reassembled in 18]  |
| 9   | 0.025103 | 74.125.127.104 | 128.208.2.151  | TCP      | 1484   | 80 → 56816 [ACK] Seq=4089 Ack=113 Win=5760 Len=1430 [TCP PDU reassembled in 18]       |
| 10  | 0.025105 | 74.125.127.104 | 128.208.2.151  | TCP      | 1484   | 80 → 56816 [ACK] Seq=5519 Ack=113 Win=5760 Len=1430 [TCP PDU reassembled in 18]       |
| 11  | 0.025106 | 74.125.127.104 | 128.208.2.151  | TCP      | 1290   | 80 → 56816 [PSH, ACK] Seq=6949 Ack=113 Win=5760 Len=1236 [TCP PDU reassembled in 18]  |
| 12  | 0.025123 | 128.208.2.151  | 74.125.127.104 | TCP      | 54     | 56816 → 80 [ACK] Seq=113 Ack=8185 Win=64240 Len=0                                     |
| 13  | 0.025203 | 74.125.127.104 | 128.208.2.151  | TCP      | 1484   | 80 → 56816 [ACK] Seq=8185 Ack=113 Win=5760 Len=1430 [TCP PDU reassembled in 18]       |
| 14  | 0.025204 | 74.125.127.104 | 128.208.2.151  | TCP      | 1484   | 80 → 56816 [ACK] Seq=9615 Ack=113 Win=5760 Len=1430 [TCP PDU reassembled in 18]       |
| 15  | 0.025211 | 128.208.2.151  | 74.125.127.104 | TCP      | 54     | 56816 → 80 [ACK] Seq=113 Ack=11045 Win=64240 Len=0                                    |
| 16  | 0.025335 | 74.125.127.104 | 128.208.2.151  | TCP      | 1290   | 80 → 56816 [PSH, ACK] Seq=11045 Ack=113 Win=5760 Len=1236 [TCP PDU reassembled in 18] |
| 17  | 0.025336 | 74.125.127.104 | 128.208.2.151  | TCP      | 1484   | 80 → 56816 [ACK] Seq=12281 Ack=113 Win=5760 Len=1430 [TCP PDU reassembled in 18]      |
| 18  | 0.025337 | 74.125.127.104 | 128.208.2.151  | HTTP     | 1281   | HTTP/1.0 200 OK (text/html)                                                           |
| 19  | 0.025349 | 128.208.2.151  | 74.125.127.104 | TCP      | 54     | 56816 → 80 [ACK] Seq=113 Ack=14939 Win=64240 Len=0                                    |
| 20  | 0.028799 | 128.208.2.151  | 74.125.127.104 | TCP      | 54     | 56816 → 80 [FIN, ACK] Seq=113 Ack=14939 Win=64240 Len=0                               |
| 21  | 0.036850 | 74.125.127.104 | 128.208.2.151  | TCP      | 60     | 80 → 56816 [ACK] Seq=14939 Ack=114 Win=5760 Len=0                                     |

Frame 4: 166 bytes on wire (1328 bits), 166 bytes captured (1328 bits) on interface 0  
 Ethernet II, Src: Dell\_d5:10:8b (00:25:64:d5:10:8b), Dst: IETF-VRRP-VRID\_01 (00:00:5e:00:01:01)  
 Internet Protocol Version 4, Src: 128.208.2.151, Dst: 74.125.127.104  
 Transmission Control Protocol, Src Port: 56816, Dst Port: 80, Seq: 1, Ack: 1, Len: 112  
 Hypertext Transfer Protocol

5. Using Display Filter, filtered seven (7) packets originating from source IP address 128.208.2.151. This includes TCP (SYN, ACK, FIN) and HTTP GET requests. The search source is consistently IP address 128.208.2.151 and the destination also same IP address of google web server 74.125.127.104.

The screenshot shows the Wireshark interface with a display filter applied: `ip.src == 128.208.2.151`. The packet list shows seven packets, with the selected packet being a TCP packet (No. 4) from 128.208.2.151 to 74.125.127.104, Seq=0, Win=64240, Len=0, MSS=1460, SACK\_PERM. The packet details pane shows the Ethernet II, Internet Protocol Version 4, and Transmission Control Protocol layers. The TCP layer shows Seq=0, Win=64240, Len=0, MSS=1460, SACK\_PERM, and the HTTP layer shows GET / HTTP/1.0.

| No. | Time     | Source        | Destination    | Protocol | Length | Info                                                           |
|-----|----------|---------------|----------------|----------|--------|----------------------------------------------------------------|
| 1   | 0.000000 | 128.208.2.151 | 74.125.127.104 | TCP      | 60     | 56816 → 80 [SYN, Seq=0 Win=64240 Len=0 MSS=1460 WS=1 SACK_PERM |
| 3   | 0.000558 | 128.208.2.151 | 74.125.127.104 | TCP      | 54     | 56816 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0                   |
| 4   | 0.000552 | 128.208.2.151 | 74.125.127.104 | HTTP     | 166    | GET / HTTP/1.0                                                 |
| 12  | 0.025123 | 128.208.2.151 | 74.125.127.104 | TCP      | 54     | 56816 → 80 [ACK] Seq=113 Ack=8185 Win=64240 Len=0              |
| 15  | 0.025211 | 128.208.2.151 | 74.125.127.104 | TCP      | 54     | 56816 → 80 [ACK] Seq=113 Ack=11045 Win=64240 Len=0             |
| 19  | 0.025349 | 128.208.2.151 | 74.125.127.104 | TCP      | 54     | 56816 → 80 [ACK] Seq=113 Ack=14939 Win=64240 Len=0             |
| 20  | 0.028799 | 128.208.2.151 | 74.125.127.104 | TCP      | 54     | 56816 → 80 [FIN, ACK] Seq=113 Ack=14939 Win=64240 Len=0        |

Frame 1: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0  
 Ethernet II, Src: Dell\_d5:10:8b (00:25:64:d5:10:8b), Dst: IETF-VRRP-VRID\_01 (00:00:5e:00:01:01)  
 Internet Protocol Version 4, Src: 128.208.2.151, Dst: 74.125.127.104  
 Transmission Control Protocol, Src Port: 56816, Dst Port: 80, Seq: 0, Len: 0

6. Using Display Filter, filtered seven (7) packets with a destination IP address 74.125.127.104. This includes TCP (SYN, ACK, FIN) and HTTP GET requests. The search destination is consistently IP address 74.125.127.104 and the source also same IP address of 128.208.2.151.

| No. | Time     | Source        | Destination    | Protocol | Length | Info                                                           |
|-----|----------|---------------|----------------|----------|--------|----------------------------------------------------------------|
| 1   | 0.000000 | 128.208.2.151 | 74.125.127.104 | TCP      | 66     | 56816 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=1 SACK_PERM |
| 3   | 0.008958 | 128.208.2.151 | 74.125.127.104 | TCP      | 54     | 56816 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0                   |
| 4   | 0.008552 | 128.208.2.151 | 74.125.127.104 | HTTP     | 166    | GET / HTTP/1.0                                                 |
| 12  | 0.025123 | 128.208.2.151 | 74.125.127.104 | TCP      | 54     | 56816 → 80 [ACK] Seq=113 Ack=8185 Win=64240 Len=0              |
| 15  | 0.025211 | 128.208.2.151 | 74.125.127.104 | TCP      | 54     | 56816 → 80 [ACK] Seq=113 Ack=11045 Win=64240 Len=0             |
| 19  | 0.025349 | 128.208.2.151 | 74.125.127.104 | TCP      | 54     | 56816 → 80 [ACK] Seq=113 Ack=14939 Win=64240 Len=0             |
| 20  | 0.028799 | 128.208.2.151 | 74.125.127.104 | TCP      | 54     | 56816 → 80 [FIN, ACK] Seq=113 Ack=14939 Win=64240 Len=0        |

|                                                                                                                                                                                                                                                                                                                                              |  |                                                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frame 1: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0<br>Ethernet II, Src: Dell_d5:10:8b (00:25:64:d5:10:8b), Dst: IETF-VRRP-VRID_01 (00:00:5e:00:01:01)<br>Internet Protocol Version 4, Src: 128.208.2.151, Dst: 74.125.127.104<br>Transmission Control Protocol, Src Port: 56816, Dst Port: 80, Seq: 0, Len: 0 |  | 0000 00 00 5e 00 01 01 00 25 64 d5 10 8b 00 00 45 00<br>0010 00 34 5b 3d 40 00 80 06 00 00 80 d0 02 97 4a 7d<br>0020 7f 68 dd f0 00 50 49 db 4d ff 00 00 00 00 80 02<br>0030 fa f0 4d 73 00 00 02 04 05 b4 01 03 03 00 01 01<br>0040 04 02 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Conclusion

This lab helps to cover the practical aspects of the packet analysis by using Wireshark. It discovers how communication begins with a three-way handshake between the client IP address and the destination IP address, and what protocols were used during the process. Using packet analysis, we can understand how the network functions and easily separate what we need and discover packets using display filters. We can use Wireshark to analyze packets in both UNIX and Windows operating systems. It can filter traffic using many criteria and create various statistics.

Performing packet analysis while investigating security incidents is always a challenging scenario because it involves the unknown element of an attacker-controlled device (Sanders, 2017). Troubleshooting is a part of packet analyzing and main part is identifying the abnormal traffic and malicious activities. The type of scanning often done first against a system is a *TCP SYN scan*, also known as a *stealth scan* or a *half-open scan* (Sanders, 2017).

## References

- Sanders, C. (2017). *Practical packet analysis* (3rd ed.).  
 Nath, A. (2015). *Packet Analysis with Wireshark*.